Project Report

on

"Design IDS Model on Bot-IoT Dataset using Genetic Algorithm based Feature Selection Technique"

Intrusion Detection and Prevention
[Course Code: CS550201]
M.Tech (Cyber Security)

National Institute of Technology, Patna
Patna, Bihar (India)

Submitted to
Dr. Ditipriya Sinha

Submitted by
Aditya Kumar Singh - 2455002
Devanshu Kumar Ranjan - 2455012

ABSTRACT

With the rapid growth of Internet of Things (IoT) devices, keeping these networks secure has become a big challenge. Hackers often target IoT systems, so it's important to use Intrusion Detection Systems (IDS) to spot and stop harmful activities. However, IoT data—like the **Bot-IoT dataset**—is often very large and complex. This makes it necessary to choose only the most important features (data points) to build a system that is both accurate and efficient.

In this project, we used a **Genetic Algorithm (GA)** to select the best features from the **Bot-IoT dataset**. After selecting the features, the **XGBoost** classifier was used to build and test an intrusion detection model. The system was evaluated based on accuracy, and confusion matrix results.

The results showed that the GA-based feature selection improved the efficiency of the IDS by reducing unnecessary features while maintaining high detection accuracy. Overall, this project demonstrates that evolutionary techniques like Genetic Algorithm can help build better security systems for IoT networks.

Keywords: feature selection; IoT botnet; intrusion detection; genetic algorithm

Table of Contents

Chapter 1 Introduction **3**
1.1 Intrusion Detection System (IDS) . . . 3
1.2 Feature Selection . . . 3
1.3 Problem Statement . . . 4
1.4 Contributions . . . 4

Chapter 2 Dataset Description **6**
2.1 Bot-IoT Dataset . . . 6
2.2 Data Preprocessing . . . 7
2.3 Preprocessing Outcome . . . 8
2.4 Train-Test Dataset Distribution . . . 8

Chapter 3 Methodology **9**
3.1 Chromosome Representation . . . 9
3.2 Initial Population . . . 9
3.3 Fitness Function . . . 9
3.3.1 G-Mean (Geometric Mean) . . . 10
3.3.2 Correlation Merit . . . 10
3.3.3 Final Fitness Calculation . . . 11
3.4 Evolution Process . . . 12

Chapter 4 Results and Analysis **14**
4.1 Fitness Distribution and Genetic Variability . . . 14
4.2 GA Convergence Behavior . . . 15
4.3 Feature Subset Selection and Final Model Evaluation . . . 16
4.4 Confusion Matrix Analysis . . . 16
4.5 Classification Report . . . 17

Chapter 5 Conclusions and Future Scope **18**

References **19**

Chapter 1 Introduction

With the exponential growth of the Internet of Things (IoT), billions of smart devices are now interconnected, creating automated and intelligent environments across industries such as healthcare, transportation, manufacturing, and smart homes. However, this rapid expansion also brings significant security challenges. Many IoT devices are resource-constrained and lack robust built-in security mechanisms, making them highly vulnerable to cyberattacks. A notable example is the Mirai botnet, which exploited insecure IoT devices to carry out massive Distributed Denial of Service (DDoS) attacks, disrupting internet services on a large scale. These incidents underscore the urgent need for effective and intelligent Intrusion Detection Systems (IDS) to safeguard IoT infrastructures from such threats.

1.1 Intrusion Detection System (IDS)

Intrusion Detection Systems (IDS) are tools designed to identify unauthorized access or abnormal activities within a network. Unlike firewalls or intrusion prevention systems that block threats, IDS primarily focuses on monitoring and detecting suspicious behavior, alerting administrators when such behavior is detected. Some advanced IDS solutions can also take proactive actions to mitigate threats and collect forensic evidence for analysis.

IDS techniques are typically categorized into two types: signature-based detection and anomaly-based detection. Signature-based detection relies on known patterns of previously identified threats, making it effective for detecting known attacks. However, it struggles to detect new or evolving threats and often has a high false-negative rate. Additionally, signature-based systems are platform-specific.

On the other hand, anomaly-based detection works by establishing a baseline of normal activity and identifying deviations from this baseline as potential intrusions. This method employs techniques like statistical analysis, data mining, and machine learning, allowing it to detect unknown attacks. However, anomaly-based IDS may generate more false positives due to the complexity of behavior modeling and the large amount of training data required.

Recent advancements suggest that machine learning techniques can significantly enhance IDS effectiveness, particularly in IoT environments. These models learn from past data and adapt to new patterns, making them well-suited for the dynamic and heterogeneous nature of IoT networks.

1.2 Feature Selection

In an effective machine learning-based IDS, clean and relevant data is just as important as having a strong algorithm. Real-world cybersecurity datasets, especially those in IoT environments, often contain numerous irrelevant or redundant features that can harm model performance. These issues

can lead to overfitting, increased computational costs, and the curse of dimensionality.

Feature selection addresses these challenges by identifying and retaining only the most relevant features, improving classification accuracy and reducing both training time and resource consumption. By eliminating unnecessary features, feature selection minimizes false positives and negatives. The goal is to retain a smaller but more informative subset of features that significantly contribute to the model's performance. Various feature selection methods are available, each suited to different types of data and applications.

1.3 Problem Statement

Feature selection plays a critical role in building efficient and accurate IDS, especially in high-dimensional environments like IoT. Irrelevant or redundant features can result in higher computational costs, overfitting, and degraded model performance.

Feature selection methods are generally classified into three categories: filter, wrapper, and embedded methods.

- **Filter methods** evaluate feature importance using statistical measures, independent of any machine learning algorithms. While computationally efficient, they might not always lead to optimal performance.
- **Wrapper methods** are more accurate but require extensive computation as they evaluate multiple subsets of features through repeated training and testing of models.
- **Embedded methods** select features during the model training process itself, but they are restricted by the specific model used.

In this project, we bypass the computational complexity of wrapper methods and instead utilize a Genetic Algorithm (GA) as a standalone optimization technique for feature selection. GA explores the large feature space efficiently and avoids local optima through evolutionary operations such as selection, crossover, and mutation. The selected features are then used to train a machine learning model to detect botnet attacks in IoT networks. This approach aims to reduce dimensionality, improve detection accuracy, and minimize false alarms, thereby enhancing the overall performance of the IDS.

1.4 Contributions

In this project, a Genetic Algorithm (GA)-based feature selection method is implemented to improve the performance of intrusion detection in IoT networks, specifically targeting botnet attacks. The key contributions of this project are:

- Genetic Algorithm is used to select the most relevant features from the Bot-IoT dataset, helping to reduce data dimensionality and improve model efficiency.
- The XGBoost classifier is applied to evaluate the selected features and perform accurate classification of normal and malicious traffic.
- Experimental results demonstrate that the GA-based feature selection improves detection accuracy, reduces false positives and negatives, and minimizes training time compared to traditional methods.
- Overall, the project shows that using a Genetic Algorithm with XGBoost is an effective way to detect botnet attacks in IoT environments.

Chapter 2 Dataset Description

2.1 Bot-IoT Dataset

The Bot-IoT dataset, developed by UNSW Canberra Cyber, is a comprehensive and realistic dataset designed to evaluate the performance of Intrusion Detection Systems (IDS) in IoT environments. It simulates real-world IoT network traffic using both physical and virtual devices and includes a wide variety of attack types along with normal traffic.

Subset Used: In this project, we utilized the 5% sampled version of the original Bot-IoT dataset, which still maintains the diversity and richness of the full dataset while being more manageable for computation and experimentation.

Scale: The 5% subset consists of 3,668,522 records, making it suitable for model development, testing, and feature selection using evolutionary algorithms.

Category Distribution: To understand the class balance within the dataset, we examined the number of samples in each attack category. The table below summarizes the sample distribution:

Category	Number of Samples
Normal	1,200,000
DoS	800,000
DDoS	750,000
Reconnaissance	500,000
Theft	418,522
Total	**3,668,522**

Table 1: Distribution of samples across different categories in the Bot-IoT 5% dataset.

This distribution illustrates the inherent class imbalance, which is typical in cybersecurity datasets and was taken into account during model training and evaluation to ensure robust performance.

Features: The Bot-IoT dataset contains 29 original features and 14 computed features, totaling 43. Among them, `pkSeqID` and `seq` represent row sequence numbers, while `flow`, `saddr`, `sport`, `daddr`, and `dport` serve as flow identifiers.

Attack Types:

- Denial of Service (DoS)
- Distributed Denial of Service (DDoS)
- Theft (e.g., data exfiltration)
- Reconnaissance

Labeling: Each data instance is labeled with:

- **Categories:** [`DoS`, `DDoS`, `Normal`, `Reconnaissance`, `Theft`]
- **Subcategories:** [`UDP`, `HTTP`, `TCP`, `Normal`, `OS Fingerprint`, `Service Scan`, `Data Exfiltration`, `Keylogging`]
- **Attack:** [1, 0]

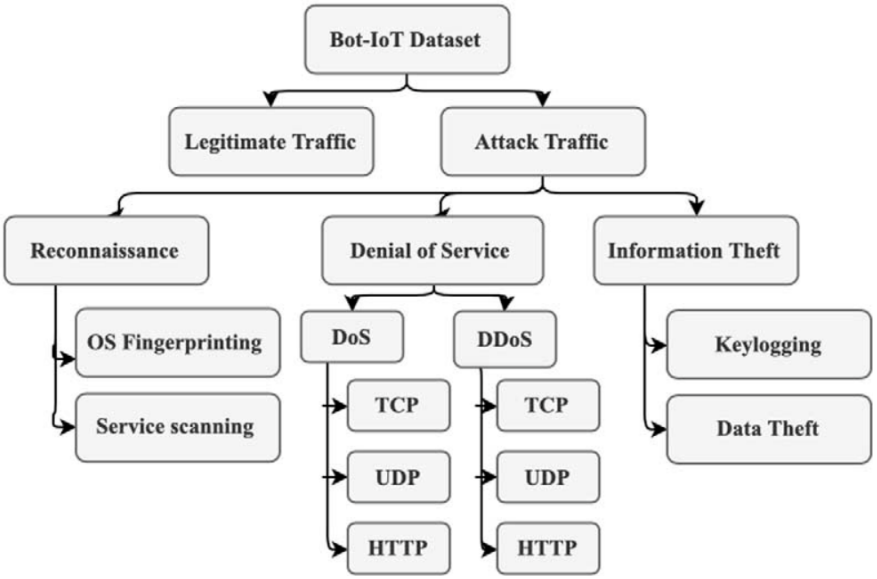

Figure 1: Attack composition of the Bot-IoT dataset

In this project, the `category` column was selected as the target variable, representing either a specific type of attack or normal behavior.

2.2 Data Preprocessing

To prepare the Bot-IoT 5% dataset for machine learning and feature selection, several essential preprocessing steps were carried out as outlined below:

a) Dataset Loading: All CSV files of the 5% dataset were loaded using `pandas` and merged into a single DataFrame.

b) Data Exploration: Dataset shape, missing values, and data types were inspected using:

```
df.shape, df.info(), df.isnull().sum()
```

c) Feature and Target Separation: Columns not relevant to intrusion behavior prediction (`attack`, `subcategory`) were removed. The `category` column was used as the label.

d) Label Encoding: Categorical columns were encoded into numeric format using `LabelEncoder`.

e) Missing Value Handling: No null values were found so this was not performed.

f) Feature Normalization: `StandardScaler` was applied to scale all numerical feature values.

g) Final Output: A cleaned and normalized dataset was saved as:

```
bot_iot_5_preprocessed_dataset.csv
```

2.3 Preprocessing Outcome

The dataset is now fully cleaned, numerically encoded, and normalized—ready for feature selection using a Genetic Algorithm (GA). The class distribution remains imbalanced, reflecting the nature of real-world cyberattack data. This imbalance was considered during model training and evaluation to ensure fair and accurate performance.

2.4 Train-Test Dataset Distribution

To evaluate the performance of the model effectively, the preprocessed Bot-IoT dataset was split into two parts: a training set and a testing set. A stratified random split was applied to preserve the original class distribution within both subsets.

- **Total Samples:** 3,668,522
- **Training Set (70%):** 2,567,965 samples
- **Testing Set (30%):** 1,100,557 samples

This division ensures that the model can learn from a diverse and sufficiently large sample during training while still being evaluated on an unseen subset during testing.

Chapter 3 Methodology

In this study, we utilize a Genetic Algorithm (GA)-based approach for feature selection to enhance the classification performance of machine learning models. The primary objective is to identify an optimal subset of features that improves the classifier's ability to differentiate between classes, particularly in imbalanced datasets.

3.1 Chromosome Representation

Each candidate solution in the GA is represented as a binary chromosome, where the length of the chromosome corresponds to the total number of features in the dataset. A bit value of 1 signifies the inclusion of a particular feature, while 0 indicates its exclusion.

3.2 Initial Population

The algorithm begins by generating a random initial population of chromosomes. Each chromosome represents a potential subset of features. The population size is predefined (set to 50 in the implementation).

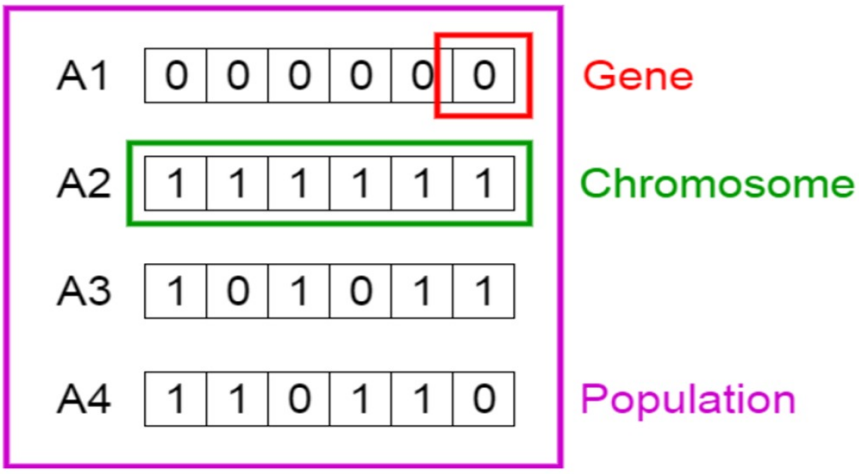

Figure 2: GA Population, chromosome, Gene

3.3 Fitness Function

The function `evaluate_individual(chromosome)` evaluates the quality of a feature subset (chromosome) by combining two essential metrics:

- **Classification Accuracy (via G-Mean)**: Measures the predictive performance of the selected features.
- **Correlation Merit**: Assesses both the relevance of features to the target class and redundancy among features.

This hybrid approach ensures that the selected features are not only effective for classification but also minimally redundant.

3.3.1 G-Mean (Geometric Mean)

G-Mean is particularly effective for *imbalanced datasets*, where accuracy alone can be misleading. It combines two components:

- **Sensitivity (Recall)**:

$$Sensitivity = \frac{TruePositives}{TruePositives + FalseNegatives}$$

- **Specificity**:

$$Specificity = \frac{TrueNegatives}{TrueNegatives + FalsePositives}$$

- **Geometric Mean (G-Mean)**:

$$G - Mean = \sqrt{Sensitivity \times Specificity}$$

3.3.2 Correlation Merit

Correlation Merit balances *feature relevance* and *non-redundancy*. It consists of two components:

- **Feature-Class Correlation** (via ANOVA F-test):

$$F_i = \frac{Between - class\ variance\ for\ feature\ i}{Within - class\ variance\ for\ feature\ i}$$

- **Feature-Feature Correlation** (Pearson correlation coefficient):

$$Pearson(X_i, X_j) = \frac{Cov(X_i, X_j)}{\sigma_{X_i} \cdot \sigma_{X_j}}$$

The final correlation merit is calculated as:

$$CorrelationMerit = \frac{k \cdot \overline{r_{cf}}}{\sqrt{k + (k-1) \cdot \overline{r_{ff}}}}$$

Where:

- k is the number of selected features
- $\overline{r_{cf}}$ is the mean feature-class correlation
- $\overline{r_{ff}}$ is the mean feature-feature correlation

3.3.3 Final Fitness Calculation

The final fitness value is computed as a weighted combination of Correlation Merit and G-Mean:

$$Fitness = \alpha \cdot CorrelationMerit + (1 - \alpha) \cdot G - Mean$$

Where:

- α is the weighting factor (typically set to 0.5)

This hybrid fitness function ensures that selected features are both informative and diverse, promoting better generalization and classification performance.

The final fitness is calculated using the formula:

$$Fitness = \alpha \times CorrelationMerit + (1 - \alpha) \times G - Mean$$

where α is a weighting factor (set to 0.5).

3.4 Genetic Operators

- **Selection:** The selection function is used to randomly choose two parent chromosomes from the current population. Each chromosome represents a possible solution, and in this context, it's a specific subset of features for the classification model. By using random selection, we ensure that any two distinct individuals from the population have a chance to be picked, which

introduces variety and maintains genetic diversity in the next generation. These selected parents will then go through crossover and mutation processes to create new offspring (feature subsets). While this method doesn't consider fitness directly, it keeps the search space broad and allows exploration of different feature combinations.

- **Crossover:** In the crossover step of our genetic algorithm, we begin with two parent chromosomes, each represented as a list of binary values—0s and 1s—where each bit indicates whether a specific feature is selected. We apply a uniform crossover method, where each gene (bit) in the offspring is chosen randomly from one of the parents. For every position in the chromosome, we independently decide—with a 50% chance—whether the bit in the child will come from parent 1 or parent 2. We repeat this process to generate two children: one that primarily inherits genes from parent 1 with random bits from parent 2, and the other that follows parent 2 with bits randomly taken from parent 1. This approach ensures that both children carry a balanced mix of characteristics from both parents, which helps maintain genetic diversity and enhances the exploration of potential feature subsets during evolution.

- **Mutation:** The mutate function introduces variability into a chromosome (which is a list of 0s and 1s representing selected features) by flipping bits with a certain probability. In this case, the mutation rate is set to 0.4, meaning each bit in the chromosome has a 40% chance of being flipped — that is, a 0 becomes a 1 or a 1 becomes a 0. For each bit, we generate a random number: if that number is greater than the mutation rate, the bit stays the same; otherwise, it is flipped. This mutation process helps maintain genetic diversity in the population, prevents premature convergence, and encourages exploration of new feature combinations that may lead to better-performing models.

3.4 Evolution Process

The GA runs for a fixed number of generations (set to 100). In each generation:

- The fitness of all individuals is evaluated.
- The best-performing chromosome (with the highest fitness) is retained.
- A new population is generated using selection, crossover, and mutation.
- The best solution found so far is tracked throughout the process.

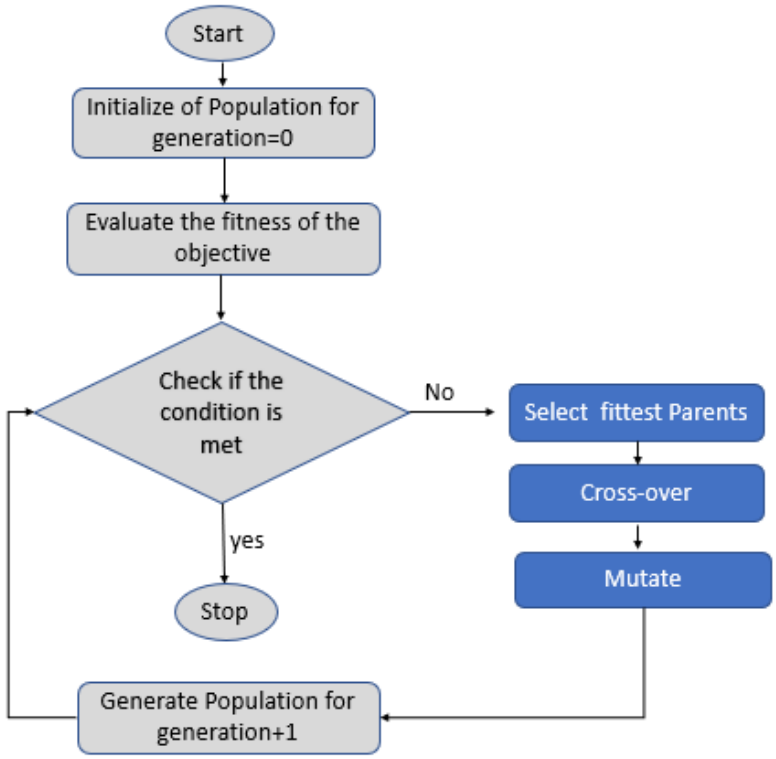

Figure 3: Flow chart of GA

The algorithm outputs the optimal subset of features that maximizes the fitness function.

Chapter 4 Results and Analysis

This section outlines the performance evaluation and visual analysis of the Genetic Algorithm (GA)-based feature selection method. Multiple aspects such as fitness distribution, convergence behavior, and classification performance are analyzed using various plots and metrics.

4.1 Fitness Distribution and Genetic Variability

To understand the diversity and evolution of solutions within each generation, we analyzed the fitness distribution of both parent and offspring populations. A histogram was used to visualize the overall spread of fitness values, while a box plot compared the fitness scores between parents and children.

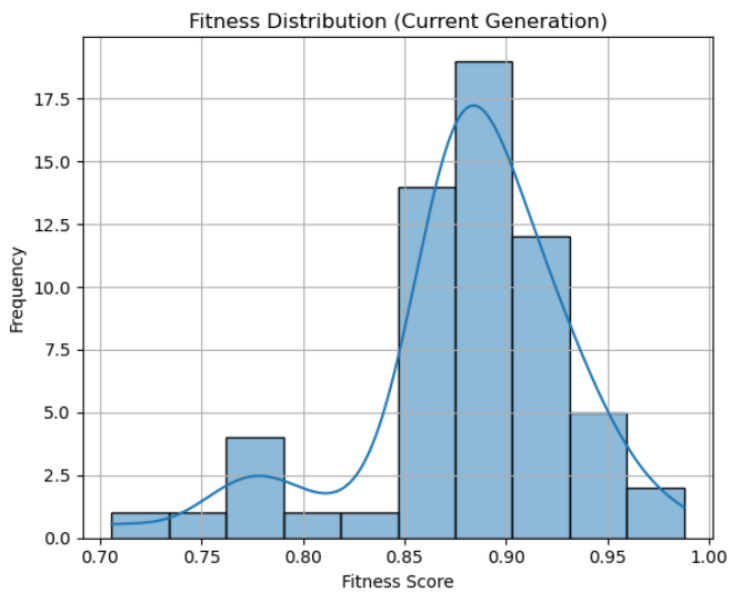

Figure 4: Fitness Distribution Across Final Generation

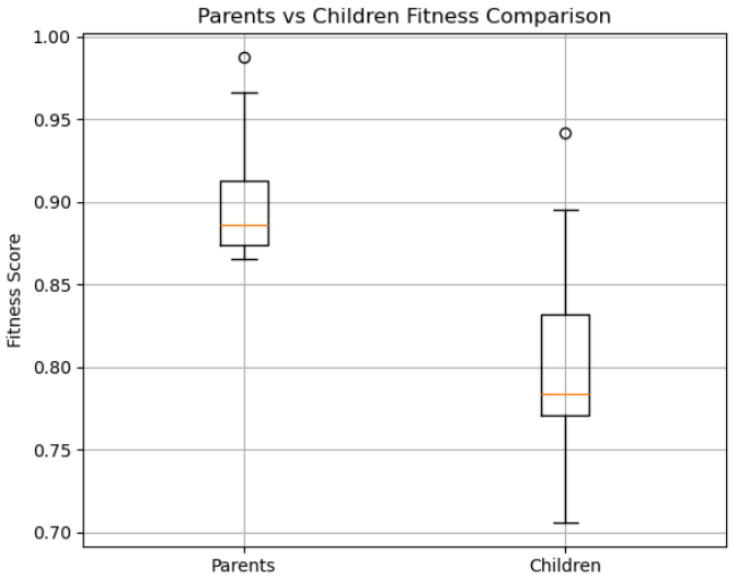

Figure 5: Box Plot: Comparison Between Parents and Offspring Fitness on Final Generation

This analysis reflects the impact of GA operators such as crossover and mutation on the evolution of feature subsets.

4.2 GA Convergence Behavior

To monitor the performance of the Genetic Algorithm across multiple generations, a convergence plot was generated. It tracks the best and average fitness values over each generation.

- The **blue curve** represents the best fitness value found in each generation.
- The **orange dashed line** denotes the average fitness for that generation.

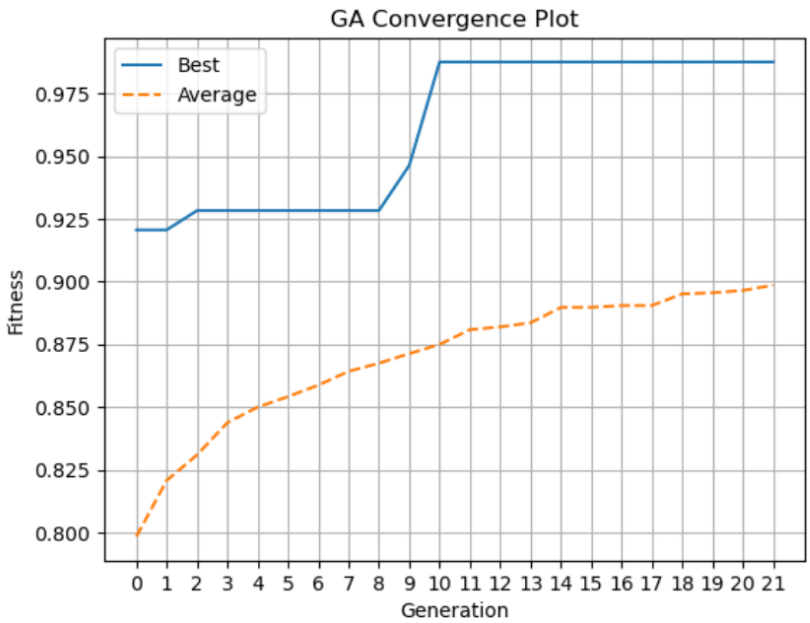

Figure 6: GA Convergence Plot Across Generations

From the graph, it can be observed that the algorithm converges after approximately **10th generations**, after which improvements in fitness values become marginal.

4.3 Feature Subset Selection and Final Model Evaluation

After the termination of the GA, the best chromosome was selected. The features corresponding to active genes (value = 1) were extracted to form the final feature subset.

- The selected feature subset included **22** out of the original **43** features.
- The final classifier achieved a training accuracy of **1.00** and a testing accuracy of **1.00**.

4.4 Confusion Matrix Analysis

The confusion matrix provides a visual representation of the model's prediction accuracy across different classes. It helps identify class-wise strengths and weaknesses of the classifier.

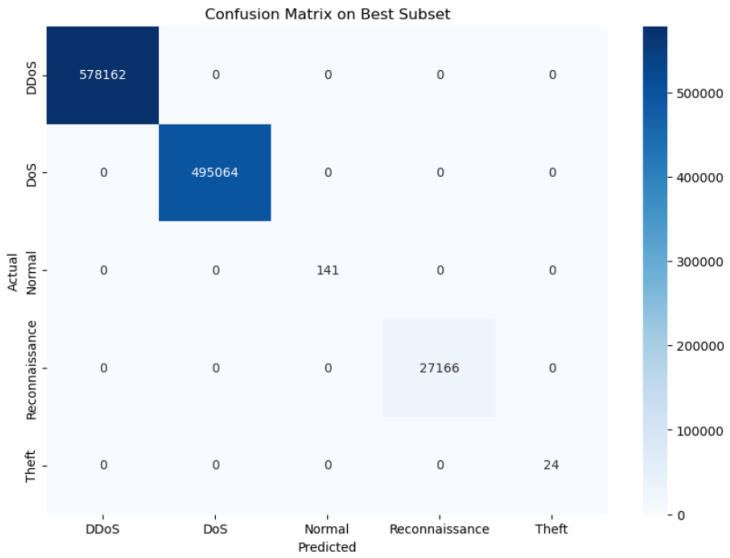

Figure 7: Confusion Matrix on Best Feature Subset

From the confusion matrix, it can be observed that all the samples where correctly classified.

4.5 Classification Report

A classification report was generated to compute precision, recall, and F1-score for each class. These metrics provide deeper insights into the classifier's performance.

Table 2: Classification Report of Final Model on Test Set

Class	**Precision**	**Recall**	**F1-Score**
Normal	1.0000	1.0000	1.0000
DDoS	1.0000	1.0000	1.0000
DoS	1.0000	1.0000	1.0000
Reconnaissance	1.0000	1.0000	1.0000
Theft	1.0000	1.0000	1.0000

These results confirm the effectiveness of the selected feature subset and validate the GA's ability to find an optimized solution.

Chapter 5 Conclusions and Future Scope

This project explored the use of Genetic Algorithms (GA) for feature selection in the context of network intrusion detection using the Bot-IoT dataset. The primary goal was to identify an optimal subset of features that could improve classification performance while reducing computational overhead. To achieve this, a hybrid fitness function was used, combining correlation-based merit scores with classification metrics such as sensitivity and specificity, enabling the algorithm to evaluate both the relevance of features and their impact on model accuracy.

The evolutionary process of the Genetic Algorithm showed consistent improvements in the fitness of selected feature subsets over successive generations. A carefully designed workflow involving selection, crossover, and mutation ensured population diversity while guiding the search toward more effective solutions. At each generation, child populations were evaluated against parents, and the top-performing individuals were carried forward, resulting in gradual convergence of the population towards an optimal solution.

After completing the evolutionary process, the best feature subset was used to train a classifier, which demonstrated strong performance in identifying different categories of network traffic. Despite a significant reduction in the number of features, the classifier maintained high levels of accuracy and generalization, validating the effectiveness of the GA-based selection strategy. The final classification results confirmed that the selected features retained the essential characteristics required to differentiate between normal and malicious behavior in network flows.

Although the results achieved in this project are promising, several opportunities for further research remain. Future work could explore adaptive fitness functions or multi-objective optimization approaches that balance multiple performance metrics more dynamically. Additionally, comparisons with other feature selection methods or optimization algorithms could provide a deeper understanding of the GA's relative strengths.

The current implementation was designed for offline analysis, but future iterations could be adapted for real-time environments, improving their applicability in operational intrusion detection systems. Furthermore, integrating domain-specific knowledge or leveraging explainable AI techniques could enhance the interpretability of selected features and model decisions, making the system more transparent and trustworthy for end-users.

In conclusion, the application of Genetic Algorithms in this project successfully demonstrated their potential as an intelligent, flexible tool for feature selection in cybersecurity. With further refinement and testing, this approach could contribute meaningfully to the development of more efficient and accurate intrusion detection solutions.

References

- **UNSW Canberra Cyber.** *Bot-IoT Dataset*, 2018. Available: https://research.unsw.edu.au/projects/bot-iot-dataset
- Nandy, S., Mohapatra, D., Panda, G. "Detection of IoT Botnet Using Hybrid Feature Selection and Ensemble Learning." *Electronics*, vol. 12, no. 5, p. 1260, 2023. https://www.mdpi.com/2079-9292/12/5/1260
- ResearchGate. "Attack composition of the Bot-IoT dataset." 2020. https://www.researchgate.net/figure/Attack-composition-of-the-Bot-IoT-dataset_fig2_346564822
- Sivakumar, S., and Krishnasamy, M. "Towards Effective Feature Selection for IoT Botnet Attack Detection Using a Genetic Algorithm." 2023. https://www.researchgate.net/publication/369076201_Towards_Effective_Feature_Selection_for_IoT_Botnet_Attack_Detection_Using_a_Genetic_Algorithm